

Frontend Interview Q&A

The questions that come up again and again on HTML, CSS, Bootstrap & Tailwind — with crisp, ready-to-say answers.

40 Questions

HTML

CSS

Bootstrap

Tailwind

By Sara Manzoor



HTML 10 questions

Q1. What is semantic HTML and why does it matter?

Using tags that describe their meaning (`<header>` , `<article>`) instead of generic `<div>` s — it improves SEO, accessibility, and code readability.

Q2. Difference between `<div>` and `<span>` ?

`<div>` is a block-level container (own line); `<span>` is inline, used for styling part of a text line.

Q3. What does the `alt` attribute do on `<img>` ?

Provides alternative text for screen readers and shows if the image fails to load — important for accessibility and SEO.

Q4. `id` vs `class` attribute?

`id` is unique per page (one element); `class` can be reused across many elements for shared styling.

Q5. What's the difference between `localStorage` , `sessionStorage` and cookies?

`localStorage` persists until cleared; `sessionStorage` clears when the tab closes; cookies are smaller, sent to the server with every request.

Q6. Why put `<script>` tags at the bottom or use `defer` ?

So the HTML/DOM finishes parsing first — it prevents render-blocking and "element not found" errors in JavaScript.

Q7. What is the DOM?

The Document Object Model — a tree-like, in-memory representation of the page that JavaScript can read and manipulate.

Q8. How do you make a page accessible to screen readers?

Use semantic tags, meaningful `alt` text, proper heading order, labelled form inputs, and `aria-*` attributes when needed.

Q9. What's the difference between `GET` and `POST` in a form?

`GET` appends data to the URL (visible, cacheable, for reads); `POST` sends data in the request body (for writes, sensitive data).

Q10. What does the `viewport` meta tag do?

`<meta name="viewport" content="width=device-width, initial-scale=1.0">` tells mobile browsers to render at the device's actual width — essential for responsive design.



CSS 10 questions

Q11. Explain the CSS box model.

Every element is a box made of content → padding → border → margin, from inside out. `box-sizing: border-box` includes padding/border in the stated width.

Q12. Difference between `relative` , `absolute` , `fixed` and `sticky` ?

`relative` offsets from its normal spot; `absolute` from nearest positioned ancestor; `fixed` from the viewport; `sticky` toggles between relative/fixed on scroll.

Q13. What is specificity and how is it calculated?

The rule that decides which CSS wins when selectors conflict. Roughly: inline style > IDs > classes/attributes > element selectors — more specific wins.

Q14. Flexbox vs Grid — when do you use each?

Flexbox for one-dimensional layouts (a single row or column); Grid for two-dimensional layouts (rows and columns together).

Q15. What is the difference between `em` and `rem` ?

`em` is relative to the parent element's font-size (compounds when nested); `rem` is always relative to the root `<html>` font-size.

Q16. How does `z-index` work?

It controls stacking order on the z-axis, but only applies to positioned elements (not `static`) — higher values sit on top.

Q17. What's a pseudo-class vs a pseudo-element?

A pseudo-class (`:hover`) targets a state of a real element; a pseudo-element (`::before`) creates/targets a virtual sub-part of it.

Q18. What is "mobile-first" responsive design?

Writing base styles for small screens first, then layering `min-width` media queries to adapt the layout as the viewport grows.

Q19. What does `display: none` do versus `visibility: hidden` ?

`display: none` removes the element from layout entirely; `visibility: hidden` hides it but still reserves its space.

Q20. What is CSS specificity's biggest beginner trap?

Overusing `!important` to override styles instead of writing more specific or better-organized selectors — it makes CSS hard to maintain.



Bootstrap 10 questions

Q21. What is Bootstrap and why use it?

A CSS/JS framework with a pre-built grid, components, and utilities — it speeds up building consistent, responsive UIs without writing custom CSS from scratch.

Q22. Explain the Bootstrap grid system.

A 12-column system: a `container` holds `row`s, and each `row` holds `col-*` elements whose widths add up to 12.

Q23. Difference between `container` and `container-fluid` ?

`container` has a fixed max-width with breakpoints; `container-fluid` always spans 100% of the viewport width.

Q24. What are Bootstrap's responsive breakpoint prefixes?

`sm` (≥576px), `md` (≥768px), `lg` (≥992px), `xl` (≥1200px), `xxl` (≥1400px) — used as `col-md-6` etc.

Q25. How do you make a Bootstrap navbar collapse on mobile?

Use `navbar-expand-lg` with a `navbar-toggler` button linked via `data-bs-toggle="collapse"` to a collapsible menu div.

Q26. What does `btn-outline-primary` vs `btn-primary` mean?

`btn-primary` is solid-filled; `btn-outline-primary` is just a colored border/text until hovered — same color family, lighter visual weight.

Q27. Can you override Bootstrap's default styles?

Yes — either with custom CSS loaded after Bootstrap, or by customizing its Sass variables before compiling, for a more maintainable approach.

Q28. What is Bootstrap's spacing utility pattern?

`{property}{sides}-{size}`, e.g. `mt-3` (margin-top), `px-2` (padding-x) — sizes run 0 to 5.

Q29. Does Bootstrap need jQuery?

Not since Bootstrap 5 — its JavaScript components now run on plain (vanilla) JavaScript, removing the jQuery dependency.

Q30. What is a common downside of using a CSS framework like Bootstrap?

Sites can look generic/templated, and unused CSS can bloat file size if you don't customize or purge what you don't need.



Tailwind CSS 10 questions

Q31. What is "utility-first" CSS?

Styling elements by composing many small, single-purpose classes (`p-4 text-center`) directly in markup, instead of writing custom named CSS classes.

Q32. Tailwind vs Bootstrap — what's the core difference?

Bootstrap ships pre-styled components (a button already looks like a button); Tailwind gives raw utilities so you design the component's look yourself.

Q33. How do responsive prefixes work in Tailwind?

Mobile-first: a plain class applies to all sizes, and prefixes like `md:` or `lg:` override it from that breakpoint upward.

Q34. What does Tailwind's "purge"/JIT engine do?

It scans your code and generates only the utility classes you actually used, keeping the final shipped CSS file extremely small.

Q35. How do you style a hover or focus state in Tailwind?

Prefix the utility: `hover:bg-blue-600` or `focus:ring-2` — variants can also be chained, like `md:hover:bg-blue-700` .

Q36. What is `@apply` used for in Tailwind?

It lets you bundle repeated utility classes into one reusable custom CSS class inside your stylesheet, for cases where markup gets too repetitive.

Q37. How does Tailwind's color naming work?

Format is `{property}-{color}-{shade}` , e.g. `bg-emerald-500` — shades run from `50` (lightest) to `900` (darkest).

Q38. What's a common criticism of Tailwind?

Markup can look cluttered with long class strings, and it has a learning curve in memorizing the utility names.

Q39. How do you handle dark mode in Tailwind?

Add the `dark:` variant (e.g. `dark:bg-gray-900`) and toggle a `dark` class on the `<html>` element.

Q40. Can Tailwind and Bootstrap be used together?

Technically yes, but it's not recommended — their utility classes and resets can conflict; most projects pick one approach.